Cryptography

Stream Ciphers and RC4

Ksawery Możdżyński

Agenda For Today

1. Encrypting byte by byte
Small data, sent at once

2. The mistake that removes the key
One rule that must never be broken

3. When the pattern repeats
A cheap generator and its cycle

4. RC4: preparing the key
Mixing a table with the key

5. RC4: producing the bytes
One byte at a time

6. RC4: the weaknesses
How it was broken

7. The end of RC4
Where it was used and what came next

Today you will need



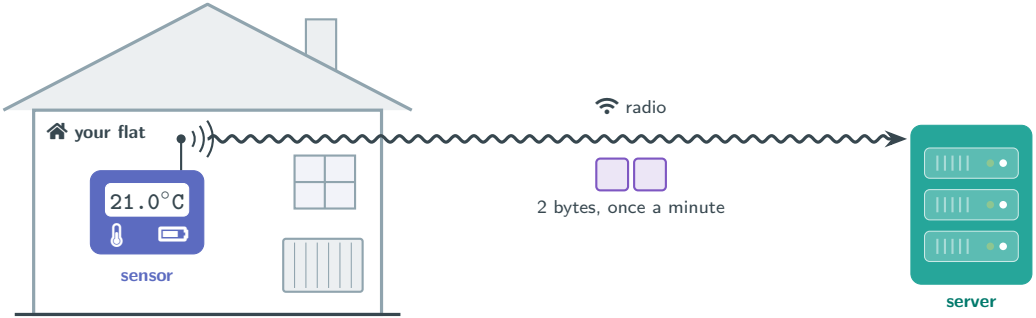
no extra libraries



any you like

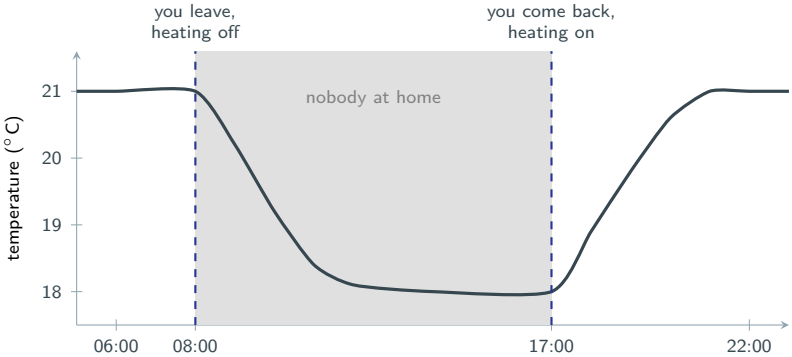
Starter files for the tasks: **GitHub**

A sensor in a flat



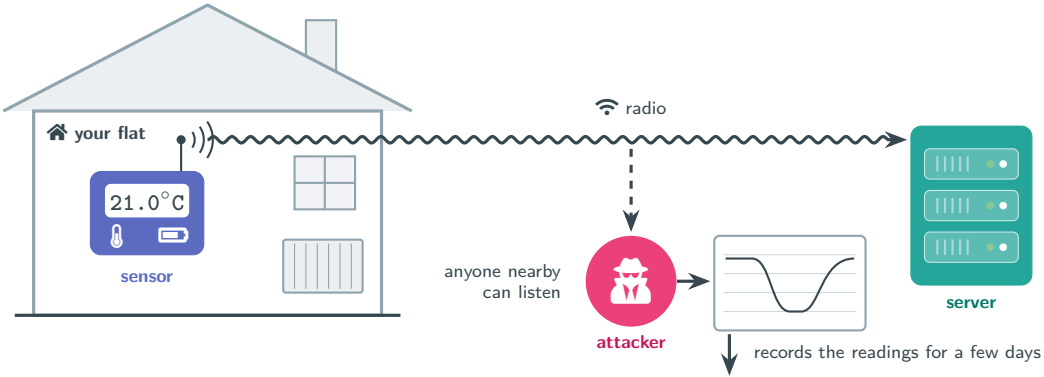
It only sends the temperature. What could go wrong?

Is the temperature a secret?



The temperature tells when nobody is at home.

Somebody is listening



Harmless readings become useful information.

So we encrypt it



The reading is the **plaintext**.



What we send instead is the **ciphertext**.



The sensor and the server share a secret **key**.

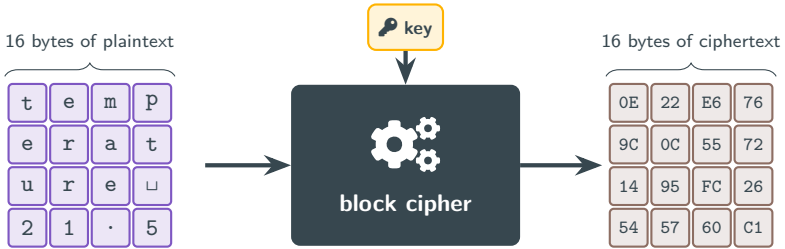


Without the key, the attacker should learn nothing.

How do we encrypt just 2 bytes?

Many ciphers work in blocks

A **block cipher** encrypts a fixed amount of data at once.

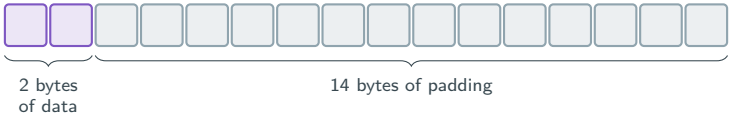


Always **16 bytes** in. Never 2, never 5.

A block cipher needs a full block of 16 bytes.

Our sensor, option 1: fill the block

Padding: extra bytes added to fill the block.



What is wrong?
We send 16 bytes when we only need 2.

Padding works, but we send more data than we need.

Our sensor, option 2: wait for a full block

The block has **16 bytes**. One reading has **2 bytes**.



Our sensor, option 2: wait for a full block

The block has **16 bytes**. One reading has **2 bytes**.



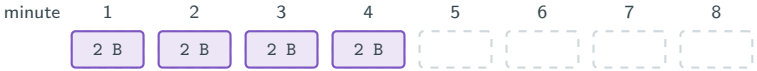
Our sensor, option 2: wait for a full block

The block has **16 bytes**. One reading has **2 bytes**.



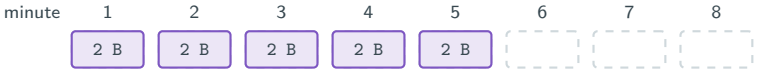
Our sensor, option 2: wait for a full block

The block has **16 bytes**. One reading has **2 bytes**.



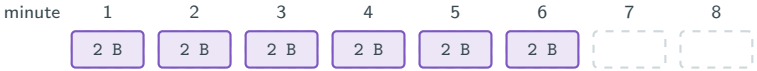
Our sensor, option 2: wait for a full block

The block has **16 bytes**. One reading has **2 bytes**.



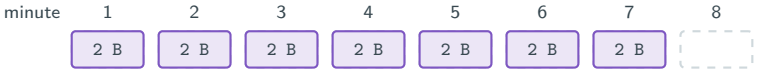
Our sensor, option 2: wait for a full block

The block has **16 bytes**. One reading has **2 bytes**.



Our sensor, option 2: wait for a full block

The block has **16 bytes**. One reading has **2 bytes**.



Our sensor, option 2: wait for a full block

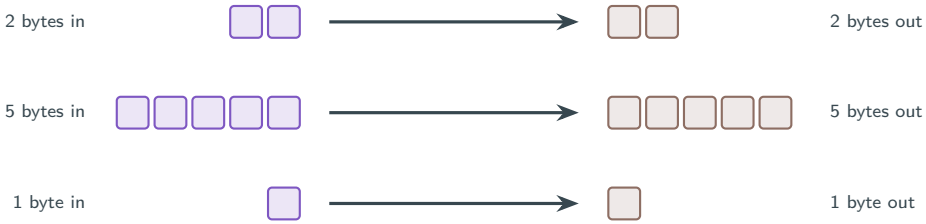
The block has **16 bytes**. One reading has **2 bytes**.



What is wrong?
No byte is wasted, but the first reading is **8 minutes old** when it leaves.
If the flat freezes or catches fire, the server finds out too late.

No bytes are wasted, but the data is late.

What do we really need?



No waiting. No padding.

We need to encrypt exactly the bytes we have, when we have them.

First try: one fixed table

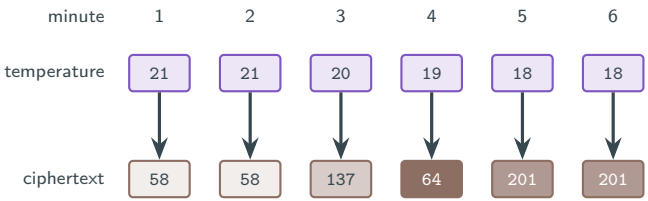
Replace every byte using one **secret table**.

plaintext byte	ciphertext byte
18	201
19	64
20	137
21	58
...	...

The table is the key. It works on single bytes.

One byte in, one byte out. Is it good enough?

First try: what goes wrong?



The attacker cannot read the numbers,
but still sees **when the readings change**,
and that the pattern **repeats every day**.

The same plaintext must not give the same ciphertext.

Why is a repeating pattern dangerous?

The attacker records the ciphertext, day after day:

	07:00	09:00	12:00	16:00	18:00	21:00
Monday	58	137	201	201	64	58
Tuesday	58	137	201	201	64	58
Wednesday	58	137	201	201	64	58
Saturday	58	58	58	58	58	58

The attacker does not know what 58 means, but sees:

Weekdays
Something changes between 9 and 18.

Saturday
Nothing changes.

The attacker does not need the numbers. The pattern is enough.

There is a better way

A fixed table is not enough. There is a better method.

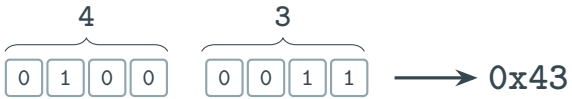
But first, let us recall three things:

1. Writing bytes in hexadecimal
2. XOR, and what happens when we use it twice
3. mod: counting around a circle

Three short reminders, then a better cipher.

Recall: writing bytes in hexadecimal

One byte = 8 bits. Too long to write, so we split it into **two halves**.



Each half is one hex digit: 0 to 9, then A to F (for 10 to 15).

`0x43 = 01000011` = the letter `C` `0xEB = 11101011`

Two hex digits are one byte.

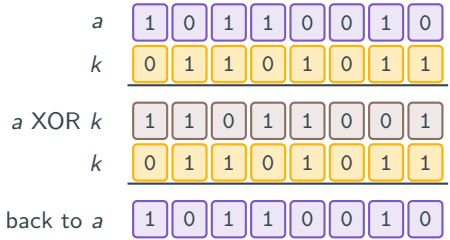
Recall: XOR

a	k	a XOR k
0	0	0
0	1	**1**
1	0	**1**
1	1	0

Different bits give **1**. Equal bits give **0**.

XOR compares two bits.

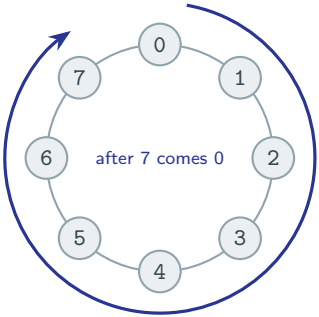
Recall: XOR twice changes nothing



In short: $(a \text{ XOR } k) \text{ XOR } k = a$

XOR with the same value twice gives back the original.

Recall: mod



mod: the remainder after dividing

`13 mod 8 = 5` (once around the circle: $13 - 8 = 5$)

`260 mod 256 = 4`

In Python

`13 % 8` `260 % 256`

mod keeps a number inside a fixed range.

Second try: a new random byte for every byte

temperature	21	21	20	19	18	18
as a byte (hex)	15	15	14	13	12	12
	XOR	XOR	XOR	XOR	XOR	XOR
random bytes	7A	C4	0E	91	5B	E2
	=	=	=	=	=	=
ciphertext	6F	D1	1A	82	49	F0

Same temperature, **different ciphertext** every time.

The pattern is gone.

One-time pad

One-time pad: XOR the plaintext with **truly random** bytes,
as many as the plaintext has, and **never use them again**.

If the rules are kept, the ciphertext says nothing about the plaintext (Shannon, 1949).

Perfect secrecy exists.

The price of the one-time pad

One year of readings:

2 bytes $\times$ 60 minutes $\times$ 24 hours $\times$ 365 days = about **1 MB**



loaded into the sensor **before it starts**
and **used up**: when it ends, the sensor stops

Perfect, and almost never practical.

The price of the one-time pad

One year of readings:

2 bytes $\times$ 60 minutes $\times$ 24 hours $\times$ 365 days $=$ about **1 MB**



loaded into the sensor **before it starts**
and **used up**: when it ends, the sensor stops

Perfect, and almost never practical.

Decision: a short key, a long keystream

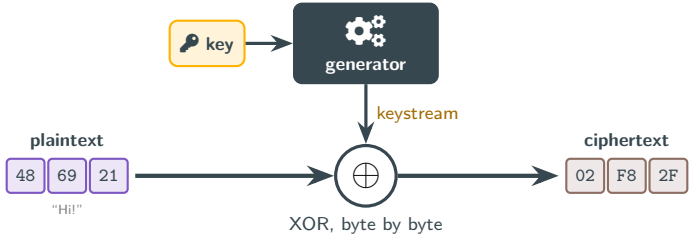


Same key → same keystream.
Without the key, nobody can predict the next byte.

We trade perfect secrecy for a key we can actually handle.

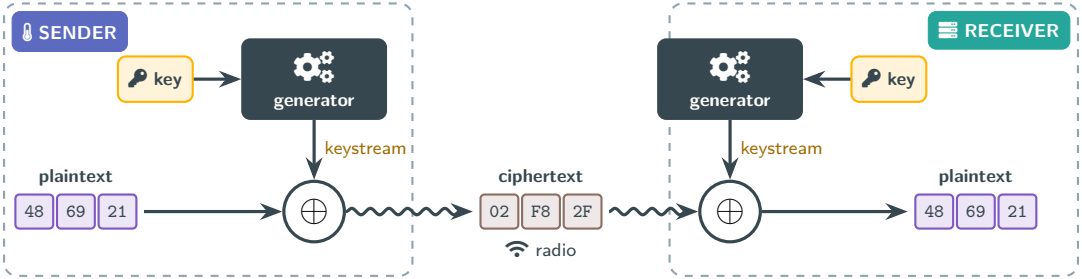
A stream cipher

Stream cipher: encrypts by XOR with a keystream, byte by byte.



Ciphertext = plaintext XOR keystream

Decrypting is the same operation



The receiver has the **same key**, so the **same keystream**.

XOR with the same keystream twice gives back the plaintext.

Task 1: decrypt a message

Task 1
You received a ciphertext and you know the keystream.

ciphertext	A3	FA	1B	ED	D8	14	BE	1A	C2	6B	2F
keystream	EB	9F	77	81	B7	34	CA	72	A7	19	4A
plaintext	?	?	?	?	?	?	?	?	?	?	?

What is the plaintext?

Task 1: what to do

```
ciphertext = [0xA3, 0xFA, 0x1B, 0xED,
              0xD8, 0x14, 0xBE, 0x1A,
              0xC2, 0x6B, 0x2F]
keystream  = [0xEB, 0x9F, 0x77, 0x81,
              0xB7, 0x34, 0xCA, 0x72,
              0xA7, 0x19, 0x4A]

# TODO: XOR the bytes in pairs,
# turn each result into a character,
# print the message
```

Task 1

Open `decrypt.py` from **GitHub**. Write the code in place of the `TODO` lines.

1. XOR each ciphertext byte with its keystream byte. In Python: `c ^ z`
2. Each result is an ASCII code. Turn it into a letter. In Python: `chr(72)` gives `H`
3. Print the whole message.

Solution: decrypt a message

```
ciphertext = [0xA3, 0xFA, 0x1B, 0xED,
              0xD8, 0x14, 0xBE, 0x1A,
              0xC2, 0x6B, 0x2F]
keystream  = [0xEB, 0x9F, 0x77, 0x81,
              0xB7, 0x34, 0xCA, 0x72,
              0xA7, 0x19, 0x4A]

plaintext = ""
for c, z in zip(ciphertext, keystream):
    plaintext += chr(c ^ z)
print(plaintext)
```

The output

`Hello there`

One byte by hand

`0xA3 XOR 0xEB = 0x48`, that is 72, that is `H`.

XOR with the keystream gives back the ASCII codes of the message.

What do we get?

Works on any number of bytes	no waiting for more data
Ciphertext as long as the plaintext	no padding
Fast	one XOR per byte, once we have the keystream
One damaged bit in transit	damages only that bit of the plaintext

Everything depends on the keystream.

Two messages, one keystream

The sensor restarts. Its generator starts again from the key.



The attacker has **C1** and **C2**, but still not the key.

Is that a problem?

The key disappears

$$C1 \text{ XOR } C2 = (P1 \text{ XOR } Z) \text{ XOR } (P2 \text{ XOR } Z)$$

The key disappears

$$\begin{aligned} \text{C1 XOR C2} &= (\text{P1 XOR Z}) \text{ XOR } (\text{P2 XOR Z}) \\ &= \text{P1 XOR P2 XOR } (\text{Z XOR Z}) \end{aligned}$$

The key disappears

$$\begin{aligned} \text{C1 XOR C2} &= (\text{P1 XOR Z}) \text{ XOR } (\text{P2 XOR Z}) \\ &= \text{P1 XOR P2 XOR } (\text{Z XOR Z}) \\ &= \text{P1 XOR P2 XOR } \boxed{0} \end{aligned}$$

The key disappears

$$
\begin{aligned}
\text{C1 XOR C2} &= (\text{P1 XOR Z}) \text{ XOR } (\text{P2 XOR Z}) \\
&= \text{P1 XOR P2 XOR } (\text{Z XOR Z}) \\
&= \text{P1 XOR P2 XOR } \boxed{0} \\
&= \mathbf{P1 \text{ XOR } P2}
\end{aligned}
$$

The keystream cancels out. The attacker never needed it.

What the attacker sees

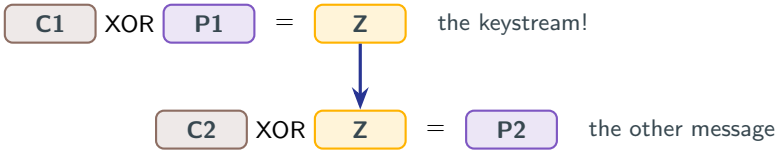
position	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
C1	A6	DA	32	D5	97	79	8F	52	E6	4D	6A	11	47	E6	0F
C2	BF	D7	32	A1	E7	7D	84	52	EE	4A	6A	1C	56	81	70
C1 XOR C2	19	0D	00	74	70	04	0B	00	08	07	00	0D	11	67	7F

00 means: both messages have **the same character** here.

Without the key, the attacker already learns about the messages.

One known message opens the other

Known plaintext: the attacker knows, or guesses, one of the messages.



Guesses are easy: file headers, greetings, fixed formats.

One known message reveals every message with the same keystream.

Task 2: one key, two messages

Task 2
Somebody carelessly encrypted two messages with the same key, so both use the same keystream.



A6DA32D597798F52E64D6A1147E60F



BFD732A1E77D8452EE4A6A1C568170

From a reliable source you learned that the first message is:

MEET ME AT 9 PM

Decrypt the second message.

Task 2: what to do

```python
c1 = bytes.fromhex("A6DA32D597798F52E64D6A1147E60F")
c2 = bytes.fromhex("BFD732A1E77D8452EE4A6A1C568170")
p1 = b"MEET ME AT 9 PM"


def xor_bytes(a, b):
    pass  # TODO 1


# TODO 2: get the keystream from c1 and p1

# TODO 3: decrypt c2 and print it
```

Task 2

Open `reuse.py` from **GitHub**. Write the code in place of the `TODO` lines.

1. Write `xor_bytes(a, b)`.
2. Get the keystream from `c1` and `p1`.
3. Decrypt `c2` and print it.

Solution: the code

```python
def xor_bytes(a, b):
    return bytes(x ^ y for x, y in zip(a, b))

keystream = xor_bytes(c1, p1)
print(xor_bytes(c2, keystream).decode())
```

The output
`THE PIN IS 4172`

`^` in Python
`x ^ y` is XOR of two bytes.

`^` is XOR in Python.

Solution: what did we learn?

- ✔ We did not use the key.
- ✔ We did not know the cipher.
- ✔ We did not break anything.

Only one mistake: **the keystream was used twice**.

This works against every stream cipher.

The rule: never twice

A keystream must **never** be used for two messages.

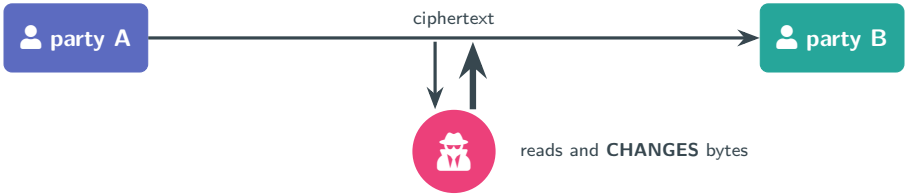
But one key always gives the same keystream:



How do we send many messages with one key? You will work it out in **Task 9**.

One keystream: one message.

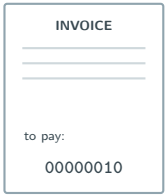
An invoice on the way



The message says how much B must pay, in PLN.

Everybody knows the format: **8 decimal digits**, as ASCII text.

00000010 means 10 PLN



The attacker cannot read it. Can the attacker change it?

First try: change a random byte

A digit in ASCII is one of 10 values: `30, 31, ... 39` (in hex).



A random ciphertext byte decrypts to a random plaintext byte:

chance of a digit: $10/256 =$ about **4%**

Almost always the result is not a number.
B sees garbage and rejects the message.

Random changes get noticed. The attacker needs a better idea.

Better: change exactly one digit

The price is probably small, so the first digit is almost surely 0.



We want 9 instead of the first 0:

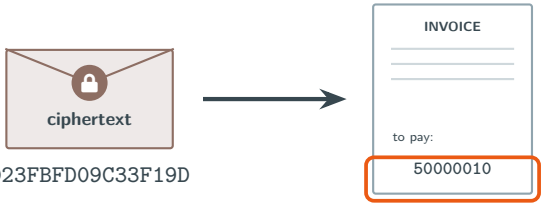
`30 XOR 39 = 09`

XOR the **first ciphertext byte** with 09.

10 PLN became 90 000 010 PLN, without the key.

Task 3: raise the price

Task 3
You intercepted an invoice for a small amount. Format: 8 decimal digits, as ASCII text.



Change the ciphertext so that party B accepts a price of exactly:

50 000 010 PLN

You do **not** know the key.

Task 3: what to do

```python
from server import receive

ciphertext = bytearray.fromhex("D23FBFD09C33F19D")

# TODO: change the ciphertext

receive(bytes(ciphertext))
```

`server.py` plays party B. Treat it as a black box.

Task 3

Open `tamper.py` from **GitHub**. Write the code in place of the `TODO` line.

1. Which byte holds the digit you want to change?
2. What must you XOR it with?
3. Run it. Party B prints the price it accepted.

Solution: raise the price

We want 5 instead of the first 0: `30 XOR 35 = 05`

```
ciphertext[0] ^= 0x05
```

Party B prints

```
Invoice accepted:  50000010 PLN
```

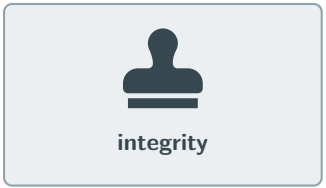
One line of code, and no key.

Encryption does not protect integrity

Integrity: the receiver can detect that the data was changed.



hides the content ✔



detects changes

Encryption hides the content.
It does not tell you whether somebody changed it.

Hiding and protecting are two different jobs.

Task 4: a cheap generator

Task 4

A company uses a stream cipher with a weak generator. Your analysis shows that its keystream is just **4 numbers** (0 to 255), repeated:



You intercepted this ciphertext:

`F6448B019B0A9108D20A9601D14FC503D84E8040DE59C5578413D44E`

You know that the plaintext starts with the name: `Anna`

1. Find the 4 numbers.
2. Decrypt the rest of the message.

Task 4: what to do

```python
ciphertext = bytes.fromhex("F6448B019B0A9108D20A9601D14F"
                           "C503D84E8040DE59C5578413D44E")
known = b"Anna"

# TODO 1: find the 4 numbers

# TODO 2: decrypt the whole ciphertext and print it
```

Task 4

Open `cheap.py` from **GitHub**. Write the code in place of the `TODO` lines.

Hint: remember mod

`i % 4` goes around 0, 1, 2, 3.

`i`	0	1	2	3	4	5	6	7	...
`i % 4`	0	1	2	3	**0**	**1**	**2**	**3**	...

Solution: a cheap generator

```
numbers = [c ^ p for c, p in zip(ciphertext, known)]

plaintext = ""
for i, c in enumerate(ciphertext):
    plaintext += chr(c ^ numbers[i % 4])
print(plaintext)
```

The output
`Anna, the safe code is 7391.`

4 known bytes
opened the whole message.

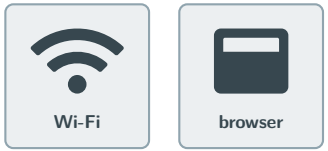
A keystream that repeats is broken. How do we build one that does not?

Meet RC4

A stream cipher from **1987**, designed by **Ron Rivest**.

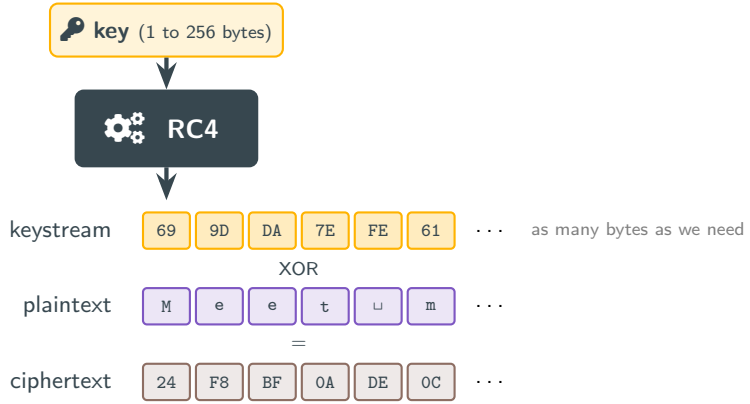
For about 20 years it protected **Wi-Fi** networks and the connection between your **browser** and a website.

We no longer use it. But it is simple, and its mistakes teach us a lot.



Short enough to fit on two slides.

RC4 from the outside



RC4 turns a short key into a long keystream.

Task 5: use RC4

```python
from rc4_ready import keystream

key = b"Kangaroo"
ciphertext = bytes.fromhex("24F8BF0ADE0C16426D343A28"
                           "F5558755720EF505F96EB7C0972B61E7")

# TODO 1: decrypt the ciphertext and print it

# TODO 2: compare keystream(key, 8) with
#         keystream(b"Kangarop", 8). How many bytes match?
```

Task 5
Open `use_rc4.py` from **GitHub**.

`rc4_ready.py`: a black box
RC4 as a ready function:
`keystream(key, n)` returns `n` bytes.

Hint
`data.hex(" ")` prints bytes as hex with spaces.

Decrypt first, then change one letter of the key.

Solution: use RC4

```
ks = keystream(key, len(ciphertext))
plaintext = bytes(c ^ k for c, k in zip(ciphertext, ks))
print(plaintext)

print(keystream(b"Kangaroo", 8).hex(" "))
print(keystream(b"Kangarop", 8).hex(" "))
```

The output

```
b'Meet me at the library at 5.'
69 9d da 7e fe 61 73 62
97 c5 8c ab f1 d1 2e 8d
```

The same bytes: 0

One letter of the key changed the whole keystream.

How does RC4 mix the key so well? Let us **open the box**.

One small change in the key changes the whole keystream.

Inside RC4: two steps

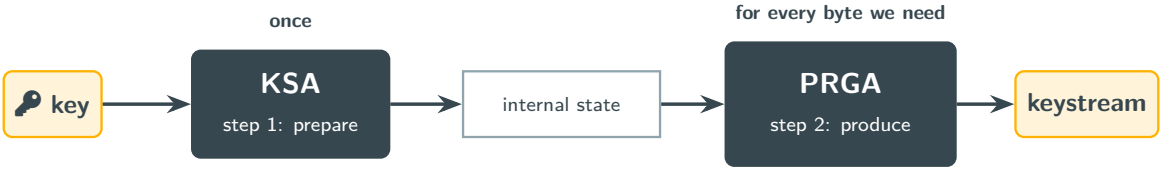
1 🔑 **Step 1: prepare**
Mix the key into the cipher's memory.
Runs **once**, before the first byte.

2 ⚙ **Step 2: produce**
Use that memory to give the next keystream byte.
Runs again **for every byte** we need.

Internal state: everything the cipher keeps in its memory between one byte and the next.

First mix the key in, then produce bytes.

Two phases



KSA: **K**ey **S**cheduling **A**lgorithm
PRGA: **P**seudo-**R**andom **G**eneration **A**lgorithm

KSA runs once. PRGA runs for every byte.

The internal state of RC4

RC4 keeps only **three variables**. This is how they start:



`S` a table of 256 bytes, at the start in order: 0, 1, 2, . . . , 255
`i`, `j` two positions in `S`. PRGA starts them at 0.

KSA uses the key to **shuffle** `S`.

Permutation: after shuffling, every number from 0 to 255 is still in `S` exactly once. Only the order changes.

256 bytes and two indexes. Nothing more.

KSA, step 1: S in order

```
key = [1, 2, 3, 4, 5]
S = list(range(256))
```

a key is just bytes: here **5** of them



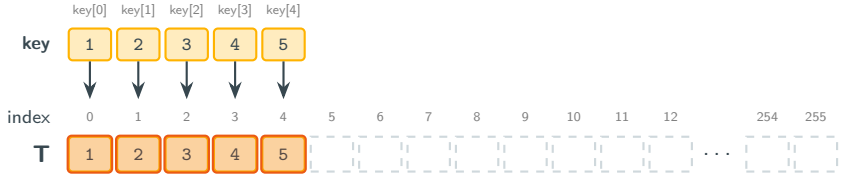
The middle of `S` is cut out. It looks the same: `S[x] = x`.

At the start, every cell holds its own index.

KSA, step 1: a helper table T

T: a helper table, used only during KSA. It holds the key, repeated.

```
T = [key[i % len(key)] for i in range(256)]    # key = [1, 2, 3, 4, 5], len(key) = 5
```

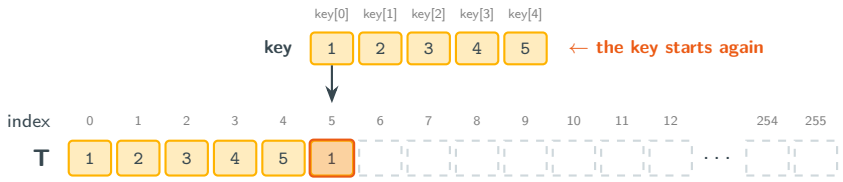



```
T[0] = key[0 % 5] = key[0] = 1    T[1] = key[1 % 5] = key[1] = 2
                    ...
T[4] = key[4 % 5] = key[4] = 5
```

The first cells of T are the key itself.

KSA, step 1: the key runs out

```
T = [key[i % len(key)] for i in range(256)]     # key = [1, 2, 3, 4, 5], len(key) = 5
```

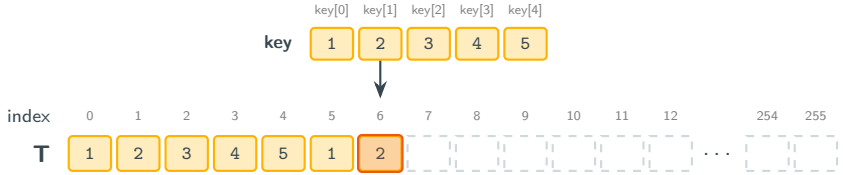


`T[5] = key[5 % 5] = key[0] = 1` ← there is no `key[5]`

When the key runs out, mod starts it again.

KSA, step 1: T[6]

```
T = [key[i % len(key)] for i in range(256)]    # key = [1, 2, 3, 4, 5], len(key) = 5
```

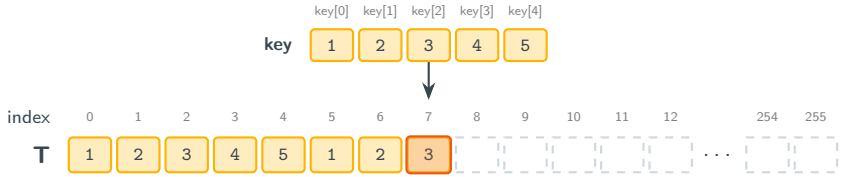



```
T[5] = key[5 % 5] = key[0] = 1
T[6] = key[6 % 5] = key[1] = 2    ← new
```

mod keeps walking around the key.

KSA, step 1: T[7]

```
T = [key[i % len(key)] for i in range(256)]    # key = [1, 2, 3, 4, 5], len(key) = 5
```




```
    T[5] = key[5 % 5] = key[0] = 1
    T[6] = key[6 % 5] = key[1] = 2
T[7] = key[7 % 5] = key[2] = 3     ← new
```

Cell i of T takes the key byte at position i mod 5.

KSA, step 1: T is full



`T[255] = key[255 % 5] = key[0] = 1`

A text key works the same way: `Moon` is the bytes `77 111 111 110`.

T is only a helper. After KSA, only S, i and j remain.

KSA, step 2: where do we swap?

`i = 0` `j = 0`




```
1  for i in range(256):
2      j = (j + S[i] + T[i]) % 256
3      S[i], S[j] = S[j], S[i]
```

`j = (0 + S[0] + T[0]) mod 256 = (0 + 0 + 1) mod 256 = 1`

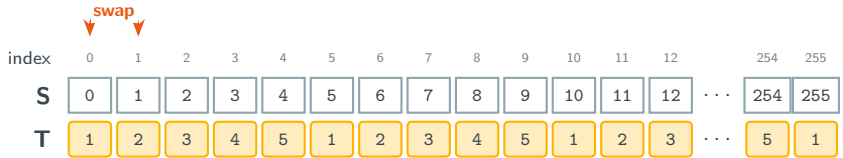
`j` depends on: its old value, `S[i]`, and the key byte `T[i]`

The key decides j.

KSA, step 2: swap

```
1  for i in range(256):
2      j = (j + S[i] + T[i]) % 256
3      S[i], S[j] = S[j], S[i]
```

`i = 0` `j = 1`: swap `S[0]` and `S[1]`



Next: S[0] and S[1] change places.

KSA, step 2: i = 0, after the swap

```
i = 0    j = 1
```



Swapping keeps every number exactly once.

KSA, step 2: i = 1

`i = 1` `j = 1` (j comes from the step before)

```
j = (j + S[1] + T[1]) mod 256 = (1 + 0 + 2) mod 256 = 3
```



Every step reads the S changed by the steps before.

KSA, step 2: i = 1, after the swap

`i = 1` `j = 3`

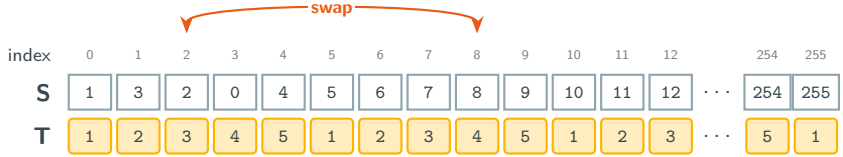


The new j = 3 is where the next step starts.

KSA, step 2: i = 2

`i = 2` `j = 3` (j comes from the step before)

`j = (j + S[2] + T[2]) mod 256 = (3 + 2 + 3) mod 256 = 8`



j can land far away from i.

KSA, step 2: i = 2, after the swap

```
i = 2    j = 8
```



Most of the middle is still in order.

KSA, step 2: i = 3

`i = 3` `j = 8` (j comes from the step before)

```
j = (j + S[3] + T[3]) mod 256 = (8 + 0 + 4) mod 256 = 12
```



Cells nobody has touched still hold their own index.

KSA, step 2: i = 3, after the swap

```
i = 3    j = 12
```



Now it is your turn.

KSA: the result after i = 255

The program repeats the same step for `i = 4, 5, ..., 255`.

After the last step: `i = 255` `j = 103`

index	0	1	2	3	4	5	6	7	8	9	10	11	12	···	254	255
S	1	3	8	201	21	27	35	67	242	145	207	89	92	···	186	88

S looks shuffled, but it is still a permutation
every number from 0 to 255 appears exactly once.

`T` is thrown away. PRGA starts with this `S` and `i = j = 0`.

Is the key really hidden in S?

Task 6: does S remember the key?

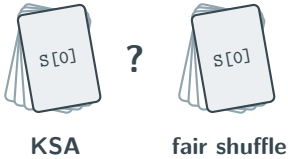
Task 6

`rc4.py` is ready: `ksa(key)` returns `S` after the KSA.

For **10 000** random keys of 16 bytes:

1. `S = ksa(key)`: count how often `S[0] == key[0]`.
2. The same with a **random shuffle** instead of the KSA: `S = list(range(256))`, then `random.shuffle(S)`.
3. Compare the two numbers.

Bonus: why? Look at the step `i = 0` on our slides.



A fair shuffle is the baseline: compare against it.

Task 6: what to do

```python
import random

from rc4 import ksa

N = 10_000
ksa_hits = 0
shuffle_hits = 0

for _ in range(N):
    key = random.randbytes(16)
    # TODO 1: S = ksa(key). Add 1 to ksa_hits if S[0] == key[0].

    # TODO 2: S = a random shuffle of the numbers 0 to 255.
    #         Add 1 to shuffle_hits if S[0] == key[0].

print(f"KSA:     {100 * ksa_hits / N:.2f}%")
print(f"shuffle: {100 * shuffle_hits / N:.2f}%")
```

Task 6

Open `remember.py` from **GitHub**. Write the code in place of the `TODO` lines.

Hint

`random.shuffle(S)` shuffles the list `S` in place.

Count the hits, then turn them into percent.

Solution: does S remember the key?

```
for _ in range(N):
    key = random.randbytes(16)
    S = ksa(key)
    ksa_hits += S[0] == key[0]
    S = list(range(256))
    random.shuffle(S)
    shuffle_hits += S[0] == key[0]
```

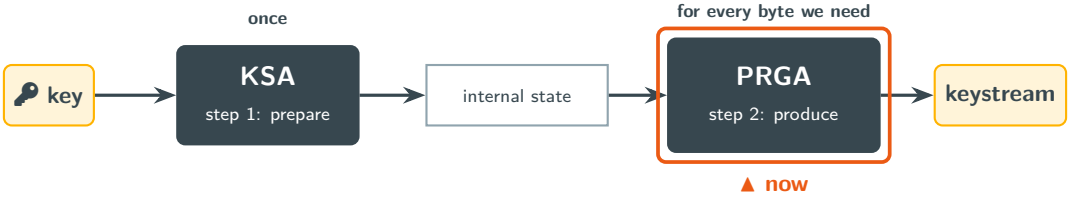


your numbers will differ a little

Why?
At `i = 0`: `j = 0 + S[0] + T[0] = key[0]`, so `S[0]` gets the value `key[0]`.
Later, `S[0]` changes only if some `j` hits 0: about 1 chance in 256 at each step.
Chance that this never happens in 255 steps: $(255/256)^{255}$ = about **37%**.

S remembers the first byte of the key.

Step 2: PRGA



Start: the S from the KSA, and `i = 0`, `j = 0`.

index	0	1	2	3	4	5	6	7	8	9	10	11	12	···	204	···	254	255
S	1	3	8	201	21	27	35	67	242	145	207	89	92	···	178	···	186	88

Every loop of PRGA gives one keystream byte.

PRGA: move i and j

```
1  i = j = 0
2  out = []
3  for _ in range(n):
4      i = (i + 1) % 256
5      j = (j + S[i]) % 256
6      S[i], S[j] = S[j], S[i]
7      t = (S[i] + S[j]) % 256
8      out.append(S[t])
```

← one loop, one keystream byte

← i walks: 1, 2, 3, . . .

← j jumps, by a number taken from S

i is predictable. j is not.

PRGA: swap

```python
i = j = 0
out = []
for _ in range(n):
    i = (i + 1) % 256
    j = (j + S[i]) % 256
    S[i], S[j] = S[j], S[i]
    t = (S[i] + S[j]) % 256
    out.append(S[t])
```

← S changes before every byte

Every output byte changes the state.

PRGA: the output byte

```
1  i = j = 0
2  out = []
3  for _ in range(n):
4      i = (i + 1) % 256
5      j = (j + S[i]) % 256
6      S[i], S[j] = S[j], S[i]
7      t = (S[i] + S[j]) % 256
8      out.append(S[t])
```

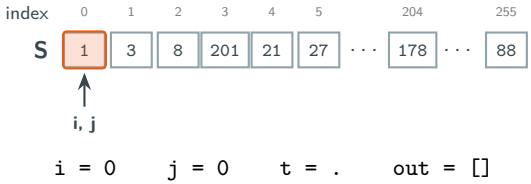
← add the two swapped numbers

← that sum is an index: output `S[t]`

The output is read from a position chosen by S itself.

PRGA, first byte: start

```
1  i = j = 0
2  out = []
3  for _ in range(n):
4      i = (i + 1) % 256
5      j = (j + S[i]) % 256
6      S[i], S[j] = S[j], S[i]
7      t = (S[i] + S[j]) % 256
8      out.append(S[t])
```

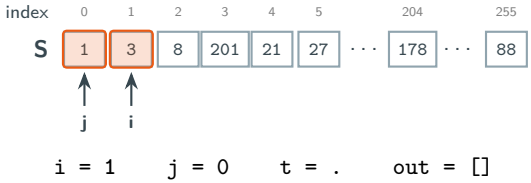


S comes from the KSA. i and j start at 0.

Start: i and j both point at cell 0.

PRGA, first byte: move i

```
1  i = j = 0
2  out = []
3  for _ in range(n):
4      i = (i + 1) % 256
5      j = (j + S[i]) % 256
6      S[i], S[j] = S[j], S[i]
7      t = (S[i] + S[j]) % 256
8      out.append(S[t])
```

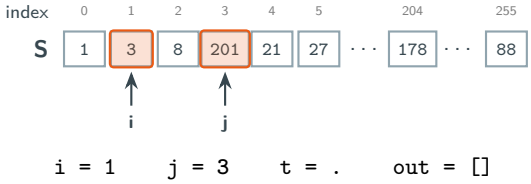


$$i = (0 + 1) \% 256 = 1$$

i moves one step forward.

PRGA, first byte: move j

```
1  i = j = 0
2  out = []
3  for _ in range(n):
4      i = (i + 1) % 256
5      j = (j + S[i]) % 256
6      S[i], S[j] = S[j], S[i]
7      t = (S[i] + S[j]) % 256
8      out.append(S[t])
```

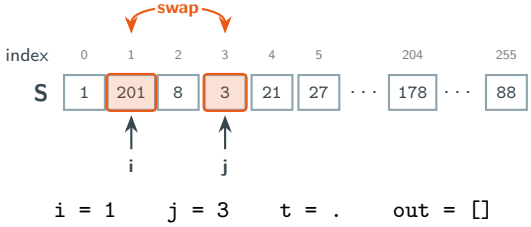


$$j = (j + S[i]) \% 256 = (0 + S[1]) \% 256 = (0 + 3) \% 256 = 3$$

j jumps by the value it finds in S[i].

PRGA, first byte: swap

```
1  i = j = 0
2  out = []
3  for _ in range(n):
4      i = (i + 1) % 256
5      j = (j + S[i]) % 256
6      S[i], S[j] = S[j], S[i]
7      t = (S[i] + S[j]) % 256
8      out.append(S[t])
```

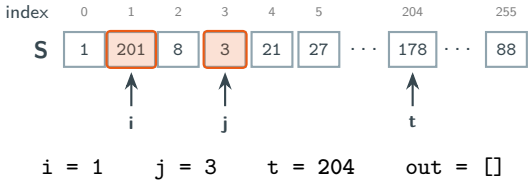


`swap S[1] and S[3]:  3 and 201 change places`

S changes before every output byte.

PRGA, first byte: compute t

```
1  i = j = 0
2  out = []
3  for _ in range(n):
4      i = (i + 1) % 256
5      j = (j + S[i]) % 256
6      S[i], S[j] = S[j], S[i]
7      t = (S[i] + S[j]) % 256
8      out.append(S[t])
```

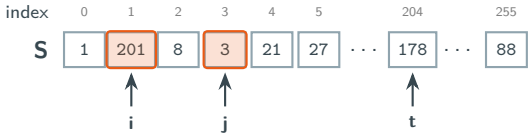


$$t = (S[i] + S[j]) \bmod 256 = (S[1] + S[3]) \bmod 256 = (201 + 3) \bmod 256 = 204$$

t is the sum of the two swapped numbers.

PRGA, first byte: output

```
1  i = j = 0
2  out = []
3  for _ in range(n):
4      i = (i + 1) % 256
5      j = (j + S[i]) % 256
6      S[i], S[j] = S[j], S[i]
7      t = (S[i] + S[j]) % 256
8      out.append(S[t])
```



S[t] = S[204] = 178 out = [178] (hex b2)

First keystream byte: **178** (hex b2).

PRGA: four bytes

i	j = (j + S[i]) mod 256	swap	t = (S[i] + S[j]) mod 256	output
1	$(0 + 3) \bmod 256 = 3$	S[1] and S[3]	$(201 + 3) \bmod 256 = 204$	178 (b2)
2	$(3 + 8) \bmod 256 = 11$	S[2] and S[11]	$(89 + 8) \bmod 256 = 97$	57 (39)
3	$(11 + 3) \bmod 256 = 14$	S[3] and S[14]	$(31 + 3) \bmod 256 = 34$	99 (63)
4	$(14 + 21) \bmod 256 = 35$	S[4] and S[35]	$(26 + 21) \bmod 256 = 47$	5 (05)

keystream: `b2` `39` `63` `05`

These are the **official test values** for this key.

Our steps give the real RC4 keystream.

All of RC4 on one slide

KSA: step 1, once

```python
def ksa(key):
    S = list(range(256))
    T = [key[i % len(key)] for i in range(256)]
    j = 0
    for i in range(256):
        j = (j + S[i] + T[i]) % 256
        S[i], S[j] = S[j], S[i]
    return S
```

```python
def keystream(key, n):
    return prga(ksa(key), n)
```

PRGA: step 2, for every byte

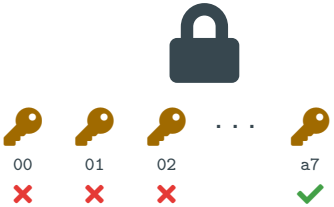
```python
def prga(S, n):
    i = j = 0
    out = []
    for _ in range(n):
        i = (i + 1) % 256
        j = (j + S[i]) % 256
        S[i], S[j] = S[j], S[i]
        t = (S[i] + S[j]) % 256
        out.append(S[t])
    return bytes(out)
```

This is the black box from Task 5.

How would you attack RC4?

The attacker has the ciphertext and knows how the message starts. The key is secret.

First idea: try every key.



Brute force: try all possible keys, one by one, until one of them gives the known start of the message.

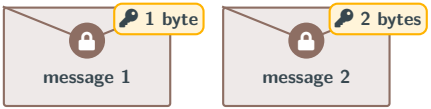
A key of *n* bytes has 256 `**` `n` possible values.

The simplest attack: try every key.

Task 7: crack a short key

Task 7

Two messages were encrypted with RC4 and very short keys. You know that every message starts with: `Hello`



the length of the key

1. Find both keys by brute force.
2. Decrypt both messages.
3. The program prints the time for each key length. Estimate the time for a key of **5 bytes**, and of **16 bytes**.

Known plaintext tells you when a guess is right.

Task 7: what to do

```python
known = b"Hello"
c1 = bytes.fromhex("F3CA21A4...")    # key: 1 byte
c2 = bytes.fromhex("C20CC3AE...")    # key: 2 bytes

def crack(ciphertext, n):
    # TODO 1: try every key of n bytes. Return a list of
    #         all keys whose keystream turns the start
    #         of the ciphertext into known.
    return []

# ready: runs crack(c1, 1) and crack(c2, 2), prints the times
# TODO 2: decrypt both messages
# TODO 3: estimate the time for 5 and for 16 bytes
```

Shortened on the slide: the hex strings and the ready loop are in the file.

Task 7

Open `crack.py` from **GitHub**.
`rc4.py` is ready: `keystream(key, n)`.

Hint: all keys of n bytes

```
for k in range(256 ** n):
    key = k.to_bytes(n, "big")
```

`256 ** n` keys: the loop visits every one of them.

Solution: crack a short key

```python
def crack(ciphertext, n):
    found = []
    for k in range(256 ** n):
        key = k.to_bytes(n, "big")
        ks = keystream(key, len(known))
        if bytes(c ^ z for c, z in zip(ciphertext, ks)) == known:
            found.append(key)
    return found
```

1 byte: key `a7`, 0.01 s

```
Hello Bob, the meeting is moved to Friday.
```

2 bytes: key `c35e`, 3 s

```
Hello Alice, the spare key is under the blue pot.
```

Only one key fits each message: 5 known bytes are enough to pick it.

Each extra byte of key: 256 times more work.

Solution: what about longer keys?

key length	keys to try	time (Python, one laptop)	
1 byte	256	0.01 s	measured
2 bytes	65 536	3 s	measured
5 bytes	about $1.1 \cdot 10^{12}$	about 1.6 years	
16 bytes	about $3.4 \cdot 10^{38}$	about $5 \cdot 10^{26}$ years	

The universe is about $1.4 \cdot 10^{10}$ years old.

Brute force does not work against a long key. **So is RC4 safe?**

A long key stops brute force. Does it stop everything?

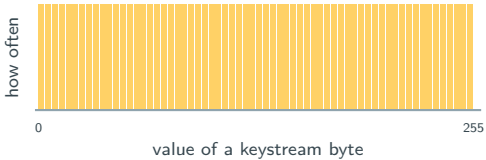
Does the keystream look random?

Uniform: every value is equally likely.

A byte has 256 possible values.

If keystream bytes are uniform, each value appears with chance

$$1/256 = \mathbf{0.39}\%$$



Let us **check** instead of trusting.

A good keystream should be uniform.

Task 8: measure the second byte

```python
for _ in range(100_000):      # 100 000 random keys
    key = random.randbytes(16)
    z = prga(ksa(key), 3)
    # TODO: count how often z[0], z[1], z[2] are 0

# print each count as a percentage
```

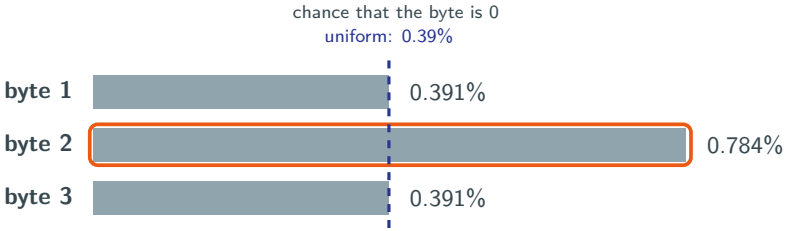
Task 8

Open `measure.py` from **GitHub**. `rc4.py` is ready: `ksa` and `prga`.



Expected: **0.39%** each.
Is it?

Solution: the second byte



10 000 000 keys. Your result with 100 000 keys is a little noisier.

The second byte is 0 about **twice** as often as it should be.

Bias

Bias: some values appear more often than they should.

RC4's **second keystream byte** has a bias towards 0.

What can an attacker do with one small bias?

A tiny bias is enough, as we will see.

The same message under many keys

One secret message is sent many times, each time with a **different key**.



Z2, **C2**, **P2**: the second byte of keystream, ciphertext, plaintext.

If **Z2 = 0**, then the ciphertext byte equals the plaintext byte:

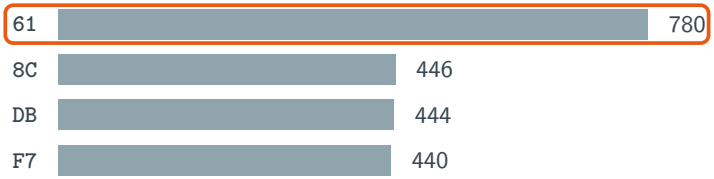
$$C2 = P2 \text{ XOR } Z2 = P2 \text{ XOR } 0 = \mathbf{P2}$$

So the most common value of C2 is the secret byte P2.

The attacker counts. The attacker needs no key.

Try it: recover a byte

100 000 messages. Secret second byte: 61 (the letter a).
Most common values of the second ciphertext byte:



The secret byte stands out, and the keys were never touched.

Task 9, step 1: the same key twice

```
key = b"our secret"   # shared with a friend
m1  = b"Meet at noon."
m2  = b"Bring the map"

c1 = rc4(key, m1)
c2 = rc4(key, m2)

# check:
xor(c1, c2) == xor(m1, m2)
rc4(key, m1) == rc4(key, m1)  # sent twice
```

Task 9

Open `one_key.py` from **GitHub**.
`rc4.py` is ready: `rc4(key, data)` encrypts and decrypts.



When is this dangerous?

One key, many messages: what can go wrong?

Solution, step 1: the same key twice

```
c1, c2 = rc4(key, m1), rc4(key, m2)
print(xor(c1, c2) == xor(m1, m2))     # True
print(rc4(key, m1) == rc4(key, m1))   # True
```

Dangerous when
the attacker knows or guesses one message: the other one opens (*The mistake that removes the key*).

Dangerous when
the same message is sent twice: the attacker sees that it repeats.

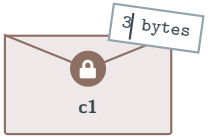
One key alone always gives the same keystream.

Task 9, step 2: a public value

Task 9, step 2

Fix it, and keep **one** secret key. For every message, make a new value of 3 bytes:

- it does not have to be secret,
- it is sent in the open, together with the ciphertext,
- it must change the ciphertext.



a public note on the outside

Check:

a) `xor(c1, c2) == xor(m1, m2)` should be `False`

b) `m1` sent twice, with two new values: are the two ciphertexts different? should be `True`

c) your friend decrypts `c1` with the key and the value

Hint

`random.randbytes(3)` gives 3 random bytes. `b"ab" + b"cd" == b"abcd"`

A public value, new for every message.

Solution, step 2: a public value

```
n1, n2 = random.randbytes(3), random.randbytes(3)
c1, c2 = rc4(n1 + key, m1), rc4(n2 + key, m2)
print(xor(c1, c2) == xor(m1, m2))               # False
print(rc4(n1 + key, m1) != rc4(n2 + key, m1))   # True
print(rc4(n1 + key, c1))                        # b'Meet at noon.'
```

RC4 has only one input, so the value has to go **into the key**.

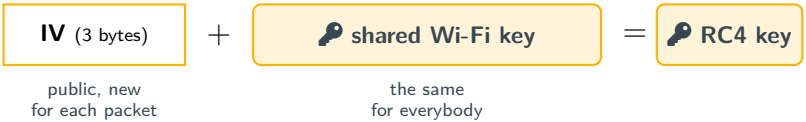
Nonce: a public value, different for every message ("**n**umber used **once**").

Is gluing the nonce to the key a good idea? **Wi-Fi did just this.**

New nonce: new keystream, with no new secret.

WEP: RC4 in Wi-Fi

WEP: the first security standard for Wi-Fi (1997).

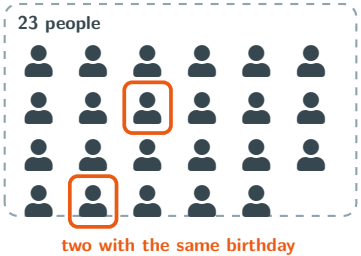


IV: initialisation vector. A nonce, under another name.

This is exactly the fix from Task 9: **nonce + key**.

WEP built a new RC4 key for every packet.

WEP, problem 1: the IV repeats



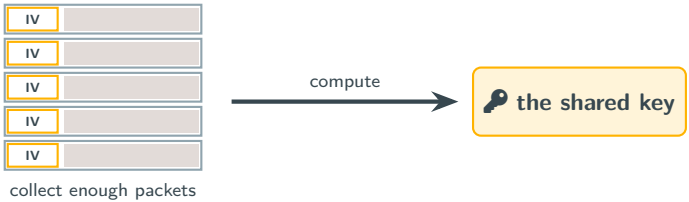
The birthday effect: in a room of 23 people, two share a birthday with a chance of about 50%. Repeats come much sooner than we expect.

IV has 3 bytes: 16 777 216 possible values
random IVs: about 50% chance of a repeat after only about **4 800 packets**

Repeated IV means repeated keystream.

WEP, problem 2: the key leaks

The public IV is the **first part** of the RC4 key.
The first keystream bytes depend strongly on the first key bytes.



Fluhrer, Mantin, Shamir, 2001.

The attacker recovers the key itself.

RC4 has no nonce



RC4 takes only a key. Everyone must invent how to add a nonce.
WEP glued it in front of the key, and that was the mistake.

RC4 makes the dangerous thing the easy thing.

How others added a nonce

RC4 takes only a key. Every protocol had to add the nonce its own way.

WEP (1997)	nonce + key	✗ broken: the nonce repeats, the key leaks
WPA (2003), TKIP mode	key, sender address and a packet counter, mixed into a new RC4 key for every packet	a quick fix for WEP, still RC4 inside
TLS	a new RC4 key for every connection, one long keystream for the whole talk	worked for years, until the biases

WPA: the fix for WEP in Wi-Fi. **TKIP**: the mode of WPA that still uses RC4.
TLS: the protocol that protects the connection between your browser and a website.

No nonce input: everyone builds it differently, and some get it wrong.

Where RC4 lived



WEP the first Wi-Fi security

WPA the quick fix for WEP, still RC4 inside (TKIP mode)

SSL / TLS the padlock in your browser (SSL is the old name)

RDP, old Windows remote desktop

Why RC4?
Simple, fast, needs no special hardware.

For years, RC4 was everywhere.

A long goodbye

1987	RC4 designed
1994	the algorithm leaks
2001	WEP key recovery; the second-byte bias
2007	WEP broken in under a minute
2013	practical attacks on RC4 inside TLS
2015	TLS standard forbids RC4 (RFC 7465)
2018	TLS 1.3 has no RC4 at all

14 years

From the first serious attack to the ban: 14 years.

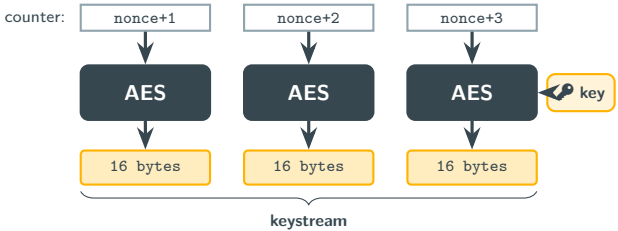
Stream and block side by side

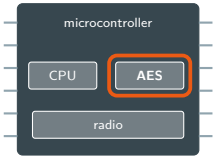
	STREAM CIPHER	BLOCK CIPHER
encrypts	any number of bytes	only full blocks
short message	nothing extra	padding, or waiting
main danger	the same keystream used twice	the same block always gives the same ciphertext

Each family has its own way to fail.

A block cipher can make a keystream

AES: the most widely used block cipher (16-byte blocks).





Counter mode: encrypt a counter and use the result as a keystream.

Many **microcontrollers** (small computers on one chip, like the one in our sensor) have AES built in as hardware: counter mode gives no waiting and no padding.

In counter mode a block cipher acts as a stream cipher.

Today: encryption and integrity together

Authenticated encryption: encrypts the data and also adds a short **tag** that lets the receiver detect changes.





Today the real question is: which authenticated cipher?

What replaced RC4

	RC4	AES-GCM, ChaCha20-Poly1305
nonce	✗ glue it yourself	✓ built in
integrity	✗ none	✓ built in
known biases	✗ yes	✓ none known

Do not use RC4 in new systems.

Summary

1. A stream cipher encrypts by XOR with a keystream.
2. Never use the same keystream twice.
3. Small and urgent: stream. Today: authenticated encryption.
4. RC4 = KSA once, then PRGA for every byte.
5. RC4 is biased and broken: know it, do not use it.